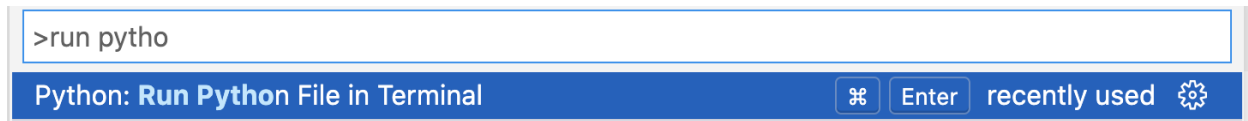


Git and Debugging

Okay, now we're ready to execute our program and help out our colleague. Using the command palette again, we can now run the program:



And so it runs...

It came back with an error! Alright, it looks like we're missing a module. We're in our conda environment so we can install the package. A quick google search tells us how to install tqdm (prefer the conda installation over the pip installation when possible):

After the installation is successful, we can run the program again.

We have an error, but it's not a module error. That's good.

We may want to share your environment with more colleagues. To allow them to quickly reproduce our environment, with all of its packages and versions, we're going to give them a copy of our environment.yml file.

This is the point where a project usually starts to drift. One person installs a package to get something working, never records it, and three weeks later nobody can reproduce the result that went into the report. Exporting the environment costs a single command and removes an entire category of argument from the project. Treat the exported file as source: commit it, review changes to it, and regenerate it whenever you add a dependency rather than editing it by hand. It is worth being deliberate about when you regenerate. If you export after every experimental install, the file churns constantly and its history stops being useful; if you never export, it drifts silently out of date and the next person to clone the repository inherits a broken environment. The habit that works is to export whenever you add a dependency you intend to keep, in the same commit as the code that needs it, so the reason for each new package is visible in the diff that introduced it. Reviewers can then see at a glance whether a dependency was added on purpose or picked up by accident while debugging something unrelated.

We do this by exporting the conda environment. If we say:

```
conda env export --from-history --file environment.yml
```

Expert user caveat: The above command is not going to work when you use pip to install anything, but it'll allow you to have environment files that work across platforms if you stick to using conda for installs.

You can learn more about exporting environments [here](#). If you want to make your environment file work across platforms, you can use the `conda env export --from-history` flag. This will only include packages that you've explicitly asked for, as opposed to including every package in your environment.

It's time to get to the next step here.

Git

We're going to use Git and Github for collaboration. What's the difference? Why don't we ask GPT3 (see [lecture 1](#) for more information of how we do that):

That's correct. I think it's possible to do lots of software engineering without understanding the basics of Git, but I think it's worth spending the time. I would first read through this page (<https://git-scm.com/book/en/v2/Getting-Started-What-is-Git%3F>). I want to highlight in particular the three main states that your files can reside in: modified, staged, and committed.

The basic Git workflow runs something like this:

1. You modify files in your working tree.
2. You selectively stage just those changes you want to be part of your next commit, which adds only those changes to the staging area.
3. You do a commit, which takes the files as they are in the staging area and stores that snapshot permanently to your Git directory.

Git is a powerful tool, and it's worth giving it some attention to get the fundamentals down. I would encourage you to read the following 3 chapters:

1. Chapter 2: <https://git-scm.com/book/en/v2/Git-Basics-Getting-a-Git-Repository>
2. Chapter 1: <https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control>
3. Chapter 3: <https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell>

Here's the cheat sheet you want to have: <https://training.github.com/downloads/github-git-cheat-sheet.pdf>

Let's put the above in action. We just created a file `environment.yml`: we're going to track the new file in a new branch. VSCode has some useful features for using directly from within the editor:

<https://code.visualstudio.com/docs/editor/versioncontrol>

On the left sidebar, you can see the source control icon. We're going to create a new branch here, called 'fix'.

Now we're in the fix branch, we're going to track the `environment.yml` file. I click on it, and then click the '+' button to stage changes.

We now see the `environment.yml` under the staged changes. Let's go ahead and add a commit message. How do we compose a Git commit message?

Here are seven rules for a proper git message, from this [guide](#):

1. Separate subject from body with a blank line
2. Limit the subject line to 60 characters
3. Capitalize the subject line
4. Do not end the subject line with a period
5. Use the imperative mood in the subject line
6. Wrap the body at 72 characters
7. Use the body to explain what and why vs. how

I've found that the use of the imperative mood in the subject line is the one that most surprises new programmers. A nice aid here is that a properly formed Git commit subject line should always be able to complete the following sentence:

If applied, this commit will your subject line here

For example:

If applied, this commit will refactor subsystem X for readability

If applied, this commit will update getting started documentation

If applied, this commit will remove deprecated methods

If applied, this commit will release version 1.0.0

Here's the example we have:

Summarize changes in around 50 characters or less

More detailed explanatory text, if necessary. Wrap it to about 72 characters or so. In some contexts, the first line is treated as the subject of the commit and the rest of the text as the body. The blank line separating the summary from the body is critical (unless

you omit the body entirely); various tools like `log`, `shortlog` and `rebase` can get confused if you run the two together.

Explain the problem that this commit is solving. Focus on why you are making this change as opposed to how (the code explains that). Are there side effects or other unintuitive consequences of this change? Here's the place to explain them.

Further paragraphs come after blank lines.

- Bullet points are okay, too

- Typically a hyphen or asterisk is used for the bullet, preceded by a single space, with blank lines in between, but conventions vary here

If you use an issue tracker, put references to them at the bottom, like this:

Resolves: #123

See also: #456, #789

In reality, you don't have most commit messages being this long, but let's try to get into the habit of following the git style rules.

A useful habit is to write the subject line first and only then decide whether the body is needed at all. If you cannot say what the commit does in fifty characters, that is usually a sign the commit is doing more than one thing.

We should be happy with this commit message (you can verify it follows all 7 rules).

We can publish the branch to github to see the new branch with changes on github. Note that if you try to push to my repository, it won't work; make sure you have [forked](#) your own repo and then push to it.

Expert User Tip: There's a VSCode extension (GitHub.vscode-pull-request-github) that also allows you to do pull requests within VSCode.

Our push to remote was successful. Here we can see the new commit in the fix branch.

Exercise:

Fork a public repo. Make a small change to the README, create a new branch and make a commit message that follows the aforementioned rules, and push it to your fork.

Debugging

We are now ready to help our colleague with their code. Last we were left with the following error:

Before diving into the following, it would be useful for you to read <https://code.visualstudio.com/docs/editor/debugging>

Now we hit the 'Run and Debug' Button. We can also get to it using F5.

Let's select the "Python file" debug configuration.

We're now presented with the above view. It looks like we have an error where we were trying to copy the path. Path should be a list, not an integer. From the left panel of local variables, we see that's not the case. paths should have been initialized using `[[startPos]]`. We can test whether we would then be able to use `path.copy()` using the debug console in the Panel bottom of screen.

Looks like that would work; so now we modify the code.